



# 汇编语言

---

## 14 多模块程序结构与宏汇编

张青

ieqzhang@zzu.edu.cn

# 实验八 模块化程序设计与宏汇编

1、编写数据输入，求平均值以及输出程序。使用有符号十进制数据输入（例5-24）、求平均值（例5-25）以及输出子程序（例5-22），编程实现从键盘输入10个数据，求平均值，并输出平均值。要求：

(a) 编写主程序文件exp0801.s。定义必要的变量和交互信息如下：

```
msg1: .string "Enter 10 numbers:\n"
```

```
msg2: .string "The mean is:"
```

调用子程序输入10个数据，求平均值，然后输出；

(b) 编写子程序exp0801s.s，包括3个子程序的过程定义；

(c) 说明进行模块连接的开发过程并上机实现；

(d) 将子程序文件形成一个子程序库(子程序库命名为libmysub.a，存放路径为Desktop/ZZUassembly/ZZUGAS/lib/libmysub.a)，说明开发过程并上机实现。

(e) 输入的10个数为：1234567890，-1234，0，1，-987654321，32767，-32768，5678，-5678，9000

# 实验八 模块化程序设计与宏汇编



2、定义一个使用逻辑指令的宏LOGICAL。编写主程序文件exp0802.s，要求：定义一个名为 LOGICAL 的宏，它代表 4条逻辑运算指令：AND、OR、XOR、TEST（严格按照这个逻辑顺序）。再定义一个名为 LOGICAL\_NOT的宏，它代表NOT逻辑运算指令。

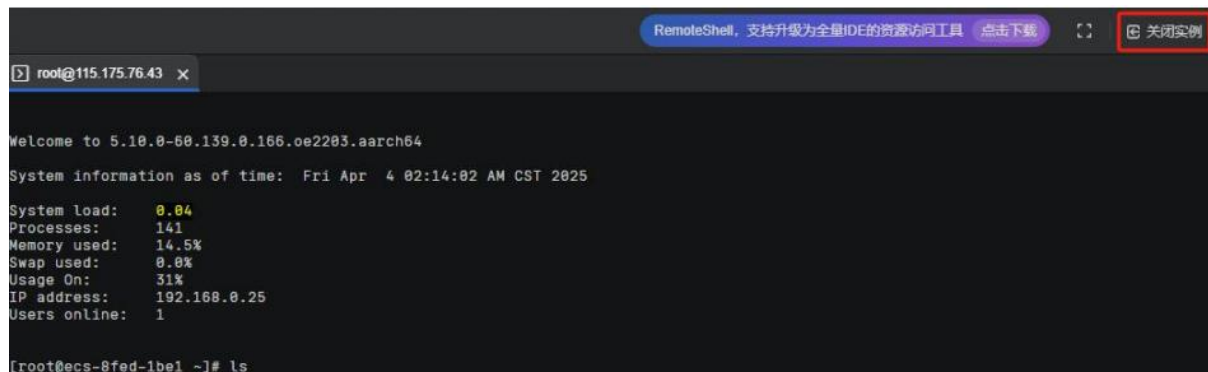
注意：由于 NOT 指令只有一个操作数，而 AND/OR/XOR/TEST 有两个操作数，需要注意参数区分。

程序内容如下，主程序不能改动，需要补充宏定义：

```
.intel_syntax noprefix
# 定义宏 (两个)
... ..
.section .text
.globl main
main:
    LOGICAL AND, 0xFF, 0x0F
    LOGICAL OR, 0xF0, 0x0F
    LOGICAL XOR, 0xFF, 0x0F
    LOGICAL TEST, 0xFF, 0x0F
    LOGICAL_NOT 0
    mov eax, 0
ret
```

# ARM64汇编程序设计（华为鲲鹏）

- 共5个实验，在华为云上完成，仅在希冀平台提交报告
  - 实验一 ARM64汇编语言开发过程
  - 实验二 分支和循环程序设计
  - 实验三 子程序设计
  - 实验四 与C语言的混合编程
  - 实验五 利用鲲鹏处理器的流水线来优化汇编代码性能
- 注意：购买时设置“按需计费”，每次实验结束后需要关闭实例、关机。



- 多模块程序结构
- 宏汇编

教材章节：5.4，5.5  
慕课：10

- ▶ 模块连接
- ▶ 子程序库
- ▶ 源文件包含
- ▶ 宏汇编

# 学习目标



- 能说出模块间共享变量或子程序，文件包含，宏定义的伪指令
- 记忆模块连接的过程和注意事项、子程序库生成和调用的过程及其注意事项，会解释文件包含的作用
- 会描述宏定义和宏调用的方法，清楚宏展开的过程和结果
- 清楚宏汇编与子程序的区别
- 会创建子程序库，会实现多模块连接，会编写宏并调用
- 会分析多模块程序的功能和执行结果，并能说明开发过程
- 能够设计多模块程序结构编写程序解决复杂问题
- 能说出程序开发过程中出现问题的原因并解决
- 养成良好的编程习惯，团队合作，互相帮助的精神

# 多模块程序结构



- 程序分段、子程序等实现了程序模块化
- 开发大型应用程序时常使用
  - 多个源程序文件
  - 目标代码模块等
- 形成多模块程序结构

## GAS支持的多模块方法

- ✓ 模块连接
- ✓ 子程序库
- ✓ 源文件包含

# 模块连接

- 子程序单独编写，汇编形成目标模块文件
- 连接时输入子程序模块文件名
- 用共用伪指令 **.global** 和外部伪指令 **.extern** 声明
  - .global 标识符** # 定义标识符的模块使用
  - .globl 标识符** # 定义标识符的模块使用
  - .extern 标识符** # 调用标识符的模块使用
- 在 GAS 中，**.extern** 通常可以省略，因为汇编器会自动处理未定义的符号。
- 子程序必须在代码段中
- 处理好子程序与主程序之间的参数传问题



# 模块连接的操作过程

- 子程序单独编写一个源程序文件
- 子程序源文件汇编形成目标模块OBJ文件
- 连接时输入子程序目标模块文件名

## 操作过程

```
gcc -c -o eg0528.o ./progs/eg0528.s
```

```
gcc -c -o eg0528s.o ./progs/eg0528s.s
```

```
gcc -o eg0528 eg0528.o eg0528s.o
```

或：

```
>gcc -o eg0528 ./progs/eg0528.S ./progs/eg0528s.S
```

```
>eg0528
```

```
RFLAGS =00000A12 O
```

# 〔例5-28〕 标志位显示程序-C语言函数调用dprf



```
//eg0528c.c
extern void dprf();
int main(){
int x=345, y=-345;
x=x+y;
dprf();
return 0;
}
```

```
>gcc -o eg0528c ./progs/eg0528c.c ./progs/eg0528s.S  
>eg0528c  
RFLAGS =00000257 C P Z
```

```
>gcc -c -o eg0528.o ./progs/eg0528c.c  
>gcc -o eg0528c eg0528.o ./progs/eg0528s.o  
>eg0528c  
RFLAGS =00000257 C P Z
```

# 模块连接练习题（学习通）



- 某程序模块使用伪指令 “.global varname” ，这表明varname\_\_\_\_\_。
  - A 是共享的，其他程序模块也可以使用
  - B 是在其他程序模块中定义的
  - C 是在本程序模块定义的
  - D 只能在本程序中使用，不能定义
  - E 既可以在本程序定义，也可以进行使用

答案： **ACE**

# 如何修改两个文件实现模块连接？(课后讨论)



## sub.asm

```
hex_in :  
...  
ret
```

## main.asm

```
.data  
....  
.text  
.global main  
main:  
call hex_in  
ret
```



---

# 子程序库

---

- 子程序库是子程序模块的集合，便于统一管理子程序
- 编写存入库文件的子程序
  - 遵循更加严格的子程序模块要求
  - 应该遵循一致的规则（以免在使用时造成混乱）
- 子程序文件编写完成、汇编形成目标模块
- 利用库管理工具程序把子程序模块加入到子程序库

# 子程序库的使用

- 子程序单独编写一个源程序文件
- 子程序源文件汇编形成目标模块文件
- 利用库管理工具把子程序模块加入到子程序库
- 在连接主程序时提供子程序库文件名

操作过程

```
ar rcs eg0528.a ./progs/eg0528s.o
```

```
gcc -o eg0528 ./progs/eg0528.s eg0528.a
```



# 子程序库练习 (学习通)



- [判断]加入子程序库的文件是子程序的源程序文件

答案：错

- [判断]存入子程序库的子程序都应该利用.global 说明

答案：对



---

# 源文件包含

---

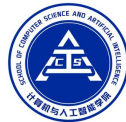
- 大型源程序可以合理地分放在若干个文本文件中
  - 各种常量定义、声明语句等组织在包含文件 (\*.inc)
  - 常用的或有价值的宏定义存放在宏定义文件 (\*.mac)
  - 常用的子程序形成汇编语言源文件 (\*.asm, \*.s)
  - 任何文本文件

➤ 使用源文件包含伪指令 `.include`

**.include** 文件名

#将指定文件内容插入主体源程序文件

## 例5-28：标志位显示程序-子程序文件



#文件名：eg0528s\_2.s，子程序

dprf：#显示标志位信息子程序

...

hdtoasc：#将eax转换为十六进制形式的字符串

...

htoasc：#将AL低4位转换为ASCII码

...

## 例5-28：标志位显示程序-主程序文件



#文件名： eg0528\_2.s, 主程序

...

**.text**                   #代码段，主程序

**.global main**

**main:**

...

**ret**

**.include "./progs/eg0528s\_2.s"**

# 源文件包含的使用

- 被包含文件
  - 文件名要符合操作系统规范
  - 只能是文本文件
  - 内容被插入源文件包含.include语句所在的位置
- 实质仍然是一个源程序
  - 只是分开在若干个文件中
  - 只需针对主体源程序文件进行汇编、连接

操作过程

```
gcc -o eg0528_2 eg0528_2.s
```

# 源文件包含练习（学习通）



- [判断]源文件包含的方式需要将多个源程序文件分别汇编

答案：错

- [判断]利用INCLUDE包含的源文件实际上只是源程序的一部分

答案：对

# 源文件包含讨论



**1.s**

```
.include 2.s  
bin_out:  
...  
ret
```

**2.s**

```
.data  
....  
.text  
.global main  
main:  
...  
ret
```





# 源文件包含讨论

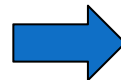


1.s

```
.include 2.s  
bin_out:  
...  
ret
```

2.s

```
.data  
....  
.text  
.global main  
main:  
...  
ret
```



1.s

```
.data  
....  
.text  
.global main  
main:  
...  
ret  
bin_out:  
...  
ret
```

使用源文件包含要注意什么问题？



---

# 宏汇编

---

- 宏 (Macro) : 具有宏名的一段汇编语句序列
- 宏需要先定义

```
.MACRO 宏名 形参表  
..... #宏定义体
```

```
.ENDM
```

- 然后程序中进行宏调用

```
宏名 实体参数
```

- 在汇编时, 宏指令被汇编程序用宏定义的代码序列替代, 实现宏展开

# 1. 宏定义和宏调用

- 宏定义

```
.macro WriteString msg  
    push rax  
    lea rax, \msg[rip]  
    call dispmsg  
    pop rax  
.endm
```

- 宏调用

```
    WriteString msg
```

- 宏展开

```
    push rax  
    lea rax, msg[rip]  
    call dispmsg  
    pop rax
```

## 【例5-29】状态标志显示宏

- 利用处理器指令PUSHF可以将当前标志寄存器内容压入堆栈，然后从堆栈弹出就能够逐个显示状态标志OF、SF、ZF、AF、PF和CF，每个标志依次对应标志寄存器的D11、D7、D6、D4、D2和D0位。将标志位移入CF、加0x30转换为ASCII码。

```
.macro rfbt bit1, bit2
    xor ebx,ebx           # EBX清0，用于保存字符
    rol eax,\bit1         # 将某个标志左移bit1位，进入最低位
    adc ebx,0x30          # 转换为ASCII字符
    mov rfmsg[rip+\bit2],bl # 保存于字符串bit2位置
.endm
```

## 〔例5-29〕



.data

rfmsg: .asciz "OF=0, SF=0, ZF=0, AF=0, PF=0, CF=0 \n" #初始化

.text

.globl main

main: # 程序执行起始位置

sub rsp, 40

mov al, 50

sub al, 80 #假设一个运算

pushfq #将标志位寄存器的内容压入堆栈

pop rax #将标志位寄存器的内容存入RAX

# 〔例5-29〕 宏调用



`rfbit 21,3`      #显示OF (原来的OF需左移21位, 进入当前CF)  
`rfbit 4,9`      #显示SF (原来的SF再左移4位, 进入当前CF)  
`rfbit 1,15`     #显示ZF (原来的ZF再左移1位, 进入当前CF)  
`rfbit 2,21`     #显示AF (原来的AF再左移2位, 进入当前CF)  
`rfbit 2,27`     #显示PF (原来的PF再左移2位, 进入当前CF)  
`rfbit 2,33`     #显示CF (原来的CF再左移2位, 进入当前CF)

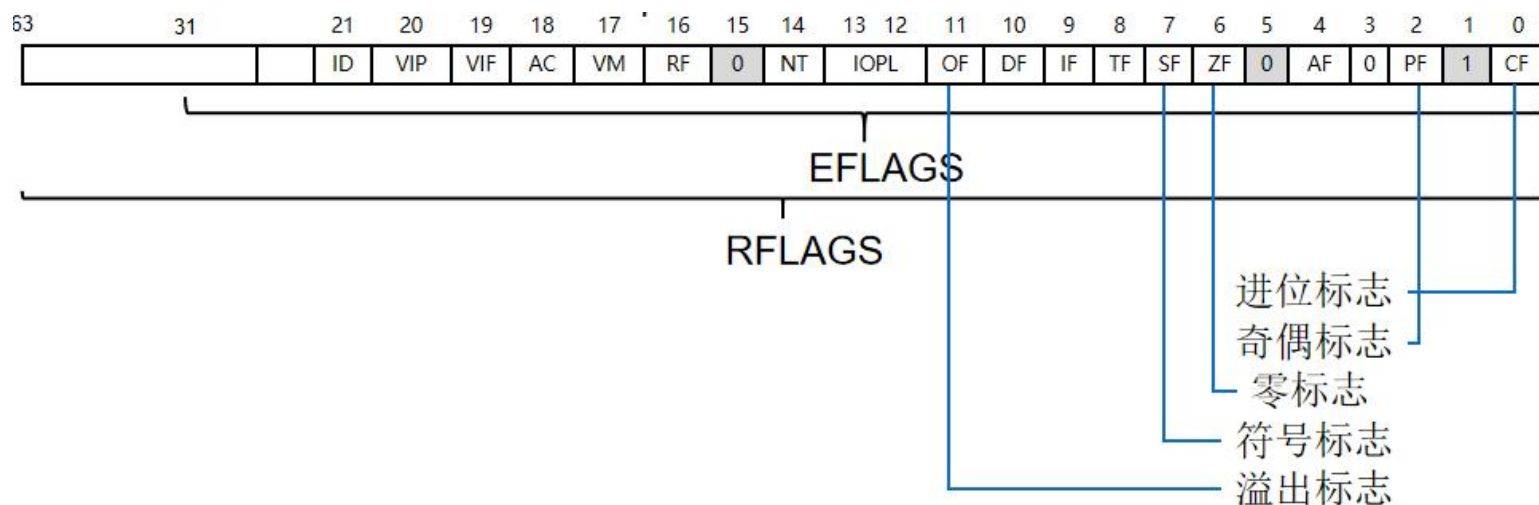
`lea rax,rfmsg[rip]`

`call dispmsg`

`mov eax,0`

`add rsp, 40`

`ret`



## 〔例5-29〕宏展开



```
27          rfbrit 21,3 #显示OF (原来的OF需左移21位, 进入当前CF)
27 000a 31DB          > xor ebx,ebx#EBX清0, 用于保存字符
27 000c C1C015        > rol eax,21#将某个标志左移BIT1位, 进入当前CF
27 000f 83D330        > adc ebx,0x30#转换为ASCII字符
27 0012 881D0300      > mov rfmsg[rip+3],bl#保存于字符串BIT2位置
27      0000

28          rfbrit 4,9 #显示SF (原来的SF再左移4位, 进入当前CF)
28 0018 31DB          > xor ebx,ebx
28 001a C1C004        > rol eax,4
28 001d 83D330        > adc ebx,0x30
28 0020 881D0900      > mov rfmsg[rip+9],bl
28      0000
```

```
gcc -Wa,-alm -g -o eg0529 ./progs/eg0529.S ./lib/winio64.a >./progs/eg0529.lst
```



## 2. 宏的参数



```
.macro rfbt bit1, bit2
    xor ebx,ebx          # EBX清0, 用于保存字符
    rol eax,\bit1        # 将某个标志左移bit1位, 进入当前CF
    adc ebx,0x30          # 转换为ASCII字符
    mov rfmsg[rip+\bit2],bl  # 保存于字符串bit2位置
.endm
```

宏调用rfbt 21,3

宏展开

宏调用时指定参数名  
rfbt bit1=21,bit2=3

```
xor ebx,ebx
rol eax,21
adc ebx,0x30
mov rfmsg[rip+3],bl
```

# 宏的参数还可以是指令中的一部分



```
#定义宏cond_jump  
.macro cond_jump cond, target  
    j\cond \target  
.endm
```

```
#宏调用  
cond_jump z, L1
```

```
#宏展开  
jz L1
```

# 宏的参数可设置默认值



```
#定义宏，设置参数size的默认值为16  
.macro alloc size=16  
    sub rsp, \size  
.endm
```

#宏调用:

```
alloc
```

#展开后的指令为:

```
sub rsp, 16
```

#宏调用:

```
alloc 32
```

#展开后的指令是:

```
sub rsp, 32
```

# 宏与子程序：简化程序



- 宏仅是源程序级的简化
  - 宏调用在汇编时进行程序语句的展开，不需要返回
  - 不减小目标程序，执行速度没有改变
- 子程序不仅简化源程序，还是目标程序级的简化
  - 子程序调用在执行时由CALL指令转向、RET指令返回
  - 形成的目标代码较短，执行速度减慢



# 宏与子程序：传递参数



- 宏通过形参、实参结合实现参数传递

- 使用软件方法，简洁直观、灵活多变
- 传递出错多体现为语法错误，易于发现



- 子程序利用寄存器、存储单元或堆栈等实现

- 运用硬件本身，规则严格、方法固定
- 传递出错常反映为逻辑或运行错误，较难排除

# 宏与子程序：选用原则



## • 选用宏

- 当程序段较短或要求较快执行时
- 宏常依附于源程序，适合进行全局性预处理



## • 选用子程序

- 当程序段较长或为减小目标代码时
- 子程序更具有独立性，可以分别编写

# 宏与子程序练习



- [判断]宏与子程序调用一样都要使用call指令实现

答案：错

- [判断]使用宏进行源程序的编写，不仅简化了源程序的编写，同时也将其生成的目标代码变得更小。

答案：错